

AI-Enabled Coding and Debugging for Corporate Placement

DAY 1: UNDERSTAND (Debugging and Code Reading Basics)

Goal: Fix broken code systematically without relying on AI, focusing on core fundamentals.

Time Slot	Module Title	Coverage and Key Topics
09:30 to 11:00	Code Execution and Common Bugs	• Common failure points in entry-level code: syntax, runtime crashes, logical errors, off-by-one errors, and boundary/edge-case failures. • Step-by-step troubleshooting loop: <i>Reproduce the error -> Trace the execution -> Isolate the broken line -> Test a fix.</i>
11:30 to 13:00	Basic Debugging and Print Logging	• Reading stack traces, compiler errors, and using basic print statements/watches to inspect variables. • Fixing classic beginner bugs: array index out of bounds, null references, broken loops, and incorrect variable mutations.
13:45 to 15:15	Working with Existing Code and Basic Testing	• Reading unfamiliar functions: tracing data inputs to expected outputs. • Understanding basic test cases: input/output verification, visible vs. hidden test cases, and ensuring fixes don't break existing functionality.
15:30 to 17:00	Hands-on Debugging Practical	• Timed exercise with 3to4 short, broken programs typical of entry-level screening assessments. • Focus on finding the root cause and making exact fixes rather than rewriting code from scratch.

DAY 2: DELEGATE (AI-Assisted Coding and Prompting)

Goal: Use AI tools to write clean, standard code faster while ensuring accuracy.

Time Slot	Module Title	Coverage and Key Topics
09:30 to 11:00	AI Coding Fundamentals	• The practical AI workflow: writing clear code prompts, asking for explanations, and reviewing generated logic before pasting. • Setting up simple AI coding extensions/chat interfaces and using them as an assistant.
11:30 to 13:00	Translating Problem Statements to AI Prompts	• Breaking complex problem statements down into clear step-by-step logic: inputs, expected output formats, constraints, and edge cases. • Preventing AI hallucination by providing precise function signatures and sample inputs.
13:45 to 15:15	Code Review and Edge Case Handling	• Verifying AI-generated code against problem constraints (e.g., handling empty inputs, negative numbers, or large loops). • Using AI to generate test cases and explain complex syntax or library methods.
15:30 to 17:00	Guided AI Coding Assessment	• Solve standard data manipulation/logic problems using AI helpers. • Students must manually test, catch edge-case failures, and fix logic gaps in AI output.

DAY 3: DELIVER (SDLC Basics and Assessment Readiness)

Goal: Build full problem solutions end-to-end and defend the logic in technical interviews.

Time Slot	Module Title	Coverage and Key Topics
09:30 to 11:00	Basic SDLC and Clean Code Standards	• Writing readable code: naming conventions, modular functions, and basic code comments. • The essential developer workflow: Write -> Test -> Refactor with AI -> Final Verification.
11:30 to 13:00	Version Control and AI Safety Nets	• Basic Git hygiene for assessment submissions: committing working states before refactoring. • Identifying common AI traps: unnecessary code complexity, wrong library calls, and bloated solutions.
13:45 to 15:15	Mock Coding Assessment	• End-to-end practical challenge modeled after typical entry-level corporate technical rounds. • Complete a feature task + fix bugs using AI tools within a timed window.
15:30 to 17:00	Code Defense and Interview Viva Preparation	• Student presentations: explaining the generated code line-by-line and justifying why the logic is correct. • Answering common interview questions: time/space complexity, edge cases, and why specific approaches were chosen over others.